

Stack

- LIFO
- A linear data structure where all the insertions and deletions are done *only at one end* rather than in the middle. This end is often called *Top* of stack.
 - **Examples:** Stack of plates, stack of coins etc.
- Applications:
 - In conversion & evaluation of arithmetic expressions in polish notations – infix, prefix & postfix.
 - In implementation of recursive procedures
 - Backtracking
 - Undo operation of a document editor or a similar environment
 - Keeping track of page-visited history of a web user
 - Reversing a list of strings

Unit II - Stack

1

Dr. Rabins Porwal

1

Stack as an ADT

- A stack of elements of any particular type is a finite sequence of elements of that type together with the following operations:
 - *Initialize* the stack to be empty.
 - Determine whether stack is *empty* or not.
 - Determine if stack is *full* or not
 - If stack is not full, then add or insert a new item at one end of the stack called Top. (This operation is called *push*)
 - If stack is not empty, then retrieve the item at its Top. (This operation is called *Peep*)
 - If the stack is not empty, then delete the item at its Top. (This operation is called *Pop*)

Unit II - Stack

2

Dr. Rabins Porwal

2

Reversing a list of strings

- Process:
 - Push each character onto the stack as it is read. When the list is finished, characters are popped off from the stack – which is accomplished in reverse order.

Unit II - Stack

3

Dr. Rabins Porwal

3

Polish Notations

- Introduced by *Jan Lukasiewicz*
- The process of writing the *operators* of an expression either before their *operands* or after them is called the *Polish notations*.
- **Property:** The order in which operations are to be performed is ascertained by the position of the operators & operands in the expression.

Unit II - Stack

4

Dr. Rabins Porwal

4

Polish Notations (contd.)

- The complex arithmetic expression can be converted into Polish strings *using stacks* which can then be executed in two operands and one operator form.
- The notations refer to these complex arithmetic expressions in three forms:
 - If the operator symbols are placed before its operands, then the expression is in *prefix notation*.
 - If the operator symbols are placed after its operands, then the expression is in *postfix notation*.
 - If the operator symbols are placed between the operands, then the expression is in *infix notations*.

Unit II - Stack

5

Dr. Rabins Porwal

5

Examples

Infix	Prefix	Postfix
$A + B$	$+ A B$	$A B +$
$(A - C) * B$	$* - A C B$	$A C - B *$
$A + (B * C)$	$+ A * B C$	$A B C * +$
$(A + B) / (C - D)$	$/ + A B - C D$	$A B + C D - /$
$(A + (B * C)) / (C - (D * B))$	$/ + A * B C - C * D B$	$A B C * + C D B * - /$

Unit II - Stack

6

Dr. Rabins Porwal

6

Conversion of Infix to Postfix Expression

- Conversion order:
 - Brackets or Parentheses
 - Exponentiation
 - Multiplications/ Division
 - Addition/ Subtraction
 - The operators with the same priority (e. g. * & /) are evaluated from L to R.
- Steps
 1. The actual evaluation is determined by inserting braces.
 2. Convert the expression in the innermost braces into postfix notation by putting the operator after the operands.
 3. Repeat the above step 2 until the entire expression is converted into postfix notation.

Unit II - Stack

7

Dr. Rabins Porwal

7

Conversion of Infix to Postfix Expression

The algorithm transforms the infix expression X into postfix form Y. This algorithm uses **stack** to temporarily hold operators and left parentheses. The postfix expression Y is constructed L→R using operands from X and operators which are removed from stack.

Unit II - Stack

8

Dr. Rabins Porwal

8

Conversion of Infix to Postfix Expression

- Algorithm
 1. Push left parenthesis "(" onto stack and add right parenthesis ")" to the end of X.
 2. Scan X from left to right and repeat steps 3 to 6 each element of X until the stack is empty.
 3. If an operand is encountered, add it to Y.
 4. If a left parenthesis is encountered, push it onto the stack.
 5. If an operator is encountered, then
 - a. Repeatedly pop from the stack and add to the right of Y each operator (on the top of stack) which has the same or higher precedence than the operator.
 - b. Add operator to stack.
 6. If a right parenthesis is encountered, then
 - a. Repeatedly pop from the stack and add to the right of Y each operator (on the top of stack) until a left parenthesis is encountered.
 - b. Remove the left parenthesis (Do not add left parenthesis to Y).

7. Exit

Unit II - Stack

9

Dr. Rabins Porwal

9

Example

- Consider the following arithmetic expression:

$$A + (B * C - (D / E \uparrow F) * G) * H$$

Postfix expression:

$$A B C * D E F \uparrow / G * - H * +$$

Unit II - Stack

10

Dr. Rabins Porwal

10

Evaluation of Postfix expression

- In an infix expression, it is difficult for the machine to keep track of precedence of operators.
- While, a postfix expression itself determines the precedence of operators (due to placement of operators).
- For a machine, it is easier to execute a postfix expression than an infix expression. As postfix expression is without parenthesis & can be evaluated as two operands and one operator at a time, this becomes easier for the compiler and the computer to handle it.

Unit II - Stack

11

Dr. Rabins Porwal

11

Algorithm to evaluate postfix expression

- Algorithm

(Reading the expression takes place from L→R)

 1. Read the next element.
 2. If element is operand, then
 - i. Push the element in the stack
 3. If element is operator, then
 - i. Pop two operands from the stack (Pop one operand in case of NOT operator)
 - ii. Evaluate the expression formed by two operands and the operator.
 - iii. Push the result of the expression in the stack.
 4. If no more elements, then
 - i. Pop the result
 5. Else
 - Go to step 1

Unit II - Stack

12

Dr. Rabins Porwal

12

Example

- Evaluate the postfix expression
 $A B + C * D /$ if $A = 2, B = 3, C = 4, D = 5$
- Ans: 4

Unit II - Stack

13

Dr. Rabins Porwal

13

Conversion of Infix to Prefix Expression

- Algorithm
 1. Push right parenthesis ")" onto stack and add left parenthesis "(" to the start of X.
 2. Scan X from right to left and repeat steps 3 to 6 for each element of X until the stack is empty.
 3. If an operand is encountered, add it to Y.
 4. If a right parenthesis is encountered, push it onto the stack.
 5. If an operator is encountered, then
 - a. Repeatedly pop from the stack and add to the left of Y each operator (on the top of stack) which has the higher precedence than the operator.
 - b. Add operator to stack.
 6. If a left parenthesis is encountered, then
 - a. Repeatedly pop from the stack and add to the left of Y each operator (on the top of stack) until a right parentheses is encountered.
 - b. Remove the right parenthesis (Do not add right parenthesis to Y).
 7. Exit

Unit II - Stack

14

Dr. Rabins Porwal

14

Example

- Consider the following arithmetic expression:
 $A \$ B * C - D + E / F / (G + G)$
 Prefix expression:
 $+ - * \$ A B C D // E F + G G$

Unit II - Stack

15

Dr. Rabins Porwal

15

Evaluation of Prefix expression

- In an infix expression, it is difficult for the machine to keep track of precedence of operators.
- While, a prefix expression itself determines the precedence of operators (due to placement of operators).
- For a machine, it is easier to execute a prefix expression than an infix expression. As prefix expression is without parenthesis & can be evaluated as two operands and one operator at a time, this becomes easier for the compiler and the computer to handle it.

Unit II - Stack

16

Dr. Rabins Porwal

16

Algorithm to evaluate prefix expression

- Algorithm
 (Reading the expression takes place from Right to Left)
 1. Read the next element.
 2. If element is operand, then
 - i. Push the element in the stack
 3. If element is operator, then
 - i. Pop two operands from the stack (Pop one operand in case of NOT operator)
 - ii. Evaluate the expression formed by two operands and the operator.
 - iii. Push the result of the expression in the stack.
 4. If no more elements, then
 - i. Pop the result
 5. Else
 Go to step 1

Unit II - Stack

17

Dr. Rabins Porwal

17

Example

- Evaluate the prefix expression
 $+ A * B + C D$ if $A = 2, B = 3, C = 4, D = 5$
- Ans: 29

Unit II - Stack

18

Dr. Rabins Porwal

18